

token

token 是采用非递归方式的正则文法(regular languages)定义的基本语法产生式(primitive productions)。Rust 源码输入可以被分解成以下几类 token：

- [关键字](#)
- [标识符](#)
- 字面量
- 生存期
- 标点符号
- 分隔符

在本文档中，“简单”token 会直接在（相关章节头部的）[字符串表产生式(production)][string table production]表单中给出，并以 monospace 字体显示。（译者注：本译作的原文中，在文法表之外的行文中也会大量出现这种直接使用简单token 来替代相关名词的做法，一般此时如果译者觉得这种 token 需要翻译时，会使用诸如：结构体(struct) 这种形式来翻译。读者需要意识到“struct”是文法里的一个 token，能以其字面形式直接出现在源码里。)

字面量是一个由单一 token（而不是由一连串 tokens）组成的表达式，它立即、直接表示它所代表的值，而不是通过名称或其他一些求值/计算规则来引用它。字面量是[常量表达式](#)的一种形式，所以它（主要）用在编译时求值。

	举例	# 号的数量	字符集	转义
字符	'H'	0	全部 Unicode	引号 & ASCII & Unicode
字符串	"hello"	0	全部 Unicode	引号 & ASCII & Unicode

原生字符串	<code>r#"hello"#</code>	0 或更多 *	全部 Unicode	N/A
字节	<code>b'H'</code>	0	全部 ASCII	引号 & 字节
字节串	<code>b"hello"</code>	0	全部 ASCII	引号 & 字节
原生字节串	<code>br#"hello"#</code>	0 或更多 *	全部 ASCII	N/A

* 字面量两侧的 # 数量必须相同。

	名称
<code>\x41</code>	7-bit 字符编码（2位数字，最大值为 0x7F）
<code>\n</code>	换行符
<code>\r</code>	回车符
<code>\t</code>	制表符
<code>\\</code>	反斜线
<code>\0</code>	Null

	名称
<code>\x7F</code>	8-bit 字符编码（2位数字）
<code>\n</code>	换行符
<code>\r</code>	回车符
<code>\t</code>	制表符
<code>\\</code>	反斜线
<code>\0</code>	Null

	名称
<code>\u{7FFF}</code>	24-bit Unicode 字符编码（最多6个数字）

	Name
<code>\'</code>	单引号
<code>\"</code>	双引号

数字字面量*	示例	指数	后缀
十进制整数	98_222	N/A	整数后缀
十六进制整数	0xff	N/A	整数后缀
八进制整数	0o77	N/A	整数后缀
二进制整数	0b1111_0000	N/A	整数后缀
浮点数	123.0E+77	Optional	浮点数后缀

* 所有数字字面量允许使用 `_` 作为可视分隔符，比如：`1_234.0E+18f64`

后缀是紧跟（无空白符）在字面量主体部分之后的非[原生标识符](#) (non-raw identifier)。

任何带有后缀的字面量（如字符串、整数等）都可以作为有效的 token，并且可以传递给宏而不会产生错误。宏自己决定如何解释这种 token，以及是否该报错。

```
macro_rules! blackhole { ($tt:tt) => () }

blackhole!("string"suffix); // OK
```

但是，最终被解析为 Rust 代码的字面量 token 上的后缀是受限制的。对于非数字字面量 token，任何后缀都最终将被弃用，而数字字面量 token 只接受下表中的后缀。

整数	浮点数
u8, i8, u16, i16, u32, i32, u64, i64, u128, i128, usize, isize	f32, f64

词法

```
CHAR_LITERAL :
    ' ( ~[' \ \n \r \t] | QUOTE_ESCAPE | ASCII_ESCAPE |
```

```
UNICODE_ESCAPE ) '
```

```
QUOTE_ESCAPE :
```

```
\' | \"
```

```
ASCII_ESCAPE :
```

```
\x OCT_DIGIT HEX_DIGIT
```

```
|\n |\r |\t |\\ |\0
```

```
UNICODE_ESCAPE :
```

```
\u{ ( HEX_DIGIT _ * )1..6 }
```

字符字面量是位于两个 `U+0027`（单引号 `'`）字符内的单个 Unicode 字符。当它是 `U+0027` 自身时，必须前置转义字符 `U+005C`（`\`）。

词法

```
STRING_LITERAL :
```

```
" (
```

```
~[" \ IsolatedCR] (译者注: IsolatedCR: 后面没有跟 \n 的
```

```
\r, 首次定义见注释)
```

```
| QUOTE_ESCAPE
```

```
| ASCII_ESCAPE
```

```
| UNICODE_ESCAPE
```

```
| STRING_CONTINUE
```

```
) * "
```

```
STRING_CONTINUE :
```

```
\ 后跟 \n
```

字符串字面量是位于两个 `U+0022`（双引号 `"`）字符内的任意 Unicode 字符序列。当它是 `U+0022` 自身时，必须前置转义字符 `U+005C`（`\`）。

字符串字面量允许换行书写。换行可以用换行符（U+000A）表示，也可以用一对回车符换行符（U+000D, U+000A）的字节序列表示。这两种字节序列通常都会被转换为 U+000A，但有例外：当换行符前置一个未转义的字符 U+005C（\）时，会导致字符 U+005C、换行符和下一行开头的所有空白符都被忽略。因此下述示例中，a 和 b 是一样的：

```
let a = "foobar";
let b = "foo\
    bar";

assert_eq!(a,b);
```

不管是字符字面量，还是非原生字符串字面量，Rust 都为其提供了额外的转义功能。转义以一个 U+005C（\）开始，并后跟如下形式之一：

- *7-bit 码点转义*以 U+0078（x）开头，后面紧跟两个十六进制数字，其最大值为 0x7F。它表示 ASCII 字符，其码值就等于字面提供的十六进制值。不允许使用更大的值，因为不能确定其是 Unicode 码点还是字节值(byte values)。
- *24-bit 码点转义*以 U+0075（u）开头，后跟多达六位十六进制数字，位于花括号 U+007B（{）和 U+007D（}）之间。这表示（需转义到的）Unicode 字符的码点等于花括号里的十六进制值。
- *空白符转义*是 U+006E（n）、U+0072（r）或者 U+0074（t）之一，依次分别表示 Unicode 码点 U+000A（LF），U+000D（CR），或者 U+0009（HT）。
- *null转义*是字符 U+0030（0），表示 Unicode 码点 U+0000（NUL）。
- *反斜线转义*是字符 U+005C（\），反斜线必须通过转义才能表示其自身。

```
RAW_STRING_LITERAL :
    r RAW_STRING_CONTENT
```

```
RAW_STRING_CONTENT :
    " ( ~ IsolatedCR ) * (非贪婪模式) "
    | # RAW_STRING_CONTENT #
```

原生字符串字面量不做任何转义。它以字符 `U+0072` (`r`) 后跟零个或多个字符 `U+0023` (`#`)，以及一个字符 `U+0022` (双引号 `"`)，这样的字符组合开始；中间原生字符串文本主体部分可包含任意的 Unicode 字符序列；再后跟另一个 `U+0022` (双引号 `"`) 字符表示文本主体结束；最后再后跟与文本主体前的那段字符组合中的同等数量的 `U+0023` (`#`) 字符。

所有包含在原生字符串文本主体中的 Unicode 字符都代表他们自身：字符 `U+0022` (双引号 `"`) (除非后跟的纯 `U+0023` (`#`) 字符串与文本主体开始前的对称相等) 或字符 `U+005C` (`\`) 此时都没有特殊含义。

字符串字面量示例：

```
"foo"; r"foo";                // foo
"\foo\""; r#"foo"##;          // "foo"

"foo #\"# bar";
r##"foo #"# bar"##;           // foo #"# bar

"\x52"; "R"; r"R";            // R
"\\x52"; r"\\x52";            // \\x52
```

BYTE_LITERAL :

b' (ASCII_FOR_CHAR | BYTE_ESCAPE) '

ASCII_FOR_CHAR :

任何 ASCII 字符 (0x00 到 0x7F) , 排除 ' , \ , \n , \r 或者 \t

BYTE_ESCAPE :

\x HEX_DIGIT HEX_DIGIT

|\n|\r|\t|\\|\0

字节字面量是单个 ASCII 字符（码值在 U+0000 到 U+007F 区间内）或一个转义字节作为字节字面量的真实主体跟在表示形式意义的字符 U+0062 (b) 和字符 U+0027 (单引号 ') 组合之后，然后再后接字符 U+0027。如果字符 U+0027 本身要出现在字面量中，它必须经由前置字符 U+005C (\) 转义。字节字面量等价于一个 u8 8-bit 无符号整型数字字面量。

词法

BYTE_STRING_LITERAL :

b" (ASCII_FOR_STRING | BYTE_ESCAPE | STRING_CONTINUE) * "

ASCII_FOR_STRING :

任何 ASCII 字符(码值位于 0x00 到 0x7F 之间), 排除 " , \ 和 IsolatedCR

非原生字节串字面量是 ASCII 字符和转义字符组成的字符序列，形式是以字符 U+0062 (b) 和字符 U+0022 (双引号 ") 组合开头，以字符 U+0022 结尾。如果字面量中包含字符 U+0022，则必须由前置的 U+005C (\) 转义。此外，字节串字面量也可以是原生字节串字面量（下面有其定义）。长度为 n 的字节串字面量类型为 &'static [u8; n]。

一些额外的转义可以在字节或非原生字节串字面量中使用，转义以 `U+005C (\)` 开始，并后跟如下形式之一：

- 字节转义以 `U+0078 (x)` 开始，后跟恰好两位十六进制数字来表示十六进制值代表的字节。
- 空白符转义是字符 `U+006E (n)`、`U+0072 (r)`，或 `U+0074 (t)` 之一，分别表示字节值 `0x0A (ASCII LF)`、`0x0D (ASCII CR)`，或 `0x09 (ASCII HT)`。
- `null` 转义是字符 `U+0030 (0)`，表示字节值 `0x00 (ASCII NUL)`。
- 反斜线转义是字符 `U+005C (\)`，必须被转义以表示其 ASCII 编码 `0x5C`。

词法

```
RAW_BYTE_STRING_LITERAL :  
    br RAW_BYTE_STRING_CONTENT
```

```
RAW_BYTE_STRING_CONTENT :  
    " ASCII* (非贪婪模式) "  
    | # RAW_BYTE_STRING_CONTENT #
```

```
ASCII :  
    任何 ASCII 字符 (0x00 到 0x7F)
```

原生字节串字面量不做任何转义。它们以字符 `U+0062 (b)` 后跟 `U+0072 (r)`，再后跟零个或多个字符 `U+0023 (#)` 及字符 `U+0022` (双引号 `"`)，这样的字符组合开始；之后是原生字节串文本主体，这部分可包含任意的 ASCII 字符序列；后跟另一个 `U+0022` (双引号 `"`) 字符表示文本主体结束；最后再后跟与文本主体前的那段字符组合中的同等数量的 `U+0023 (#)` 字符。原生字节串字面量不能包含任何非 ASCII 字节。

原生字节串文本主体中的所有字符都代表它们自身的 ASCII 编码，字符 `U+0022`（双引号 `"`）（除非后跟的纯 `U+0023`（`#`）字符串与文本主体开始前的对称相等）或字符 `U+005C`（`\`）此时都没有特殊含义。

字节串字面量示例：

```
b"foo"; br"foo";                // foo
b"\\"foo\\""; br#"\"foo\""#;    // "\"foo\""

b"foo #"\"# bar";
br##"foo #"\"# bar"##;          // foo #"\"# bar

b"\x52"; b"R"; br"R";           // R
b"\\x52"; br"\\x52";            // \\x52
```

数字字面量可以是整型字面量，也可以是浮点型字面量，识别这两种字面量的文法是混合在一起的。

词法

INTEGER_LITERAL :

(DEC_LITERAL | BIN_LITERAL | OCT_LITERAL | HEX_LITERAL)
INTEGER_SUFFIX?

DEC_LITERAL :

DEC_DIGIT (DEC_DIGIT|_)*

BIN_LITERAL :

0b (BIN_DIGIT|_)* BIN_DIGIT (BIN_DIGIT|_)*

OCT_LITERAL :

0o (OCT_DIGIT|_)* OCT_DIGIT (OCT_DIGIT|_)*

HEX_LITERAL :

```
0x (HEX_DIGIT|_)* HEX_DIGIT (HEX_DIGIT|_)*
```

```
BIN_DIGIT : [0-1]
```

```
OCT_DIGIT : [0-7]
```

```
DEC_DIGIT : [0-9]
```

```
HEX_DIGIT : [0-9 a-f A-F]
```

```
INTEGER_SUFFIX :
```

```
    u8 | u16 | u32 | u64 | u128 | usize  
    | i8 | i16 | i32 | i64 | i128 | isize
```

整型字面量具备下述 4 种形式之一：

- 十进制字面量以十进制数字开头，后跟十进制数字和*下划线(`_`)*的任意组合。
- 十六进制字面量以字符序列 `U+0030 U+0078` (`0x`) 开头，后跟十六进制数字和下划线的任意组合（至少一个数字）。
- 八进制字面量以字符序列 `U+0030 U+006F` (`0o`) 开头，后跟八进制数字和下划线的任意组合（至少一个数字）。
- 二进制字面量以字符序列 `U+0030 U+0062` (`0b`) 开头，后跟二进制数字和下划线的任意组合（至少一个数字）。

与其它字面量一样，整型字面量后面可紧跟一个整型后缀，该后缀强制设定了字面量的数据类型。整型后缀须为如下整型类型之一：`u8`、`i8`、`u16`、`i16`、`u32`、`i32`、`u64`、`i64`、`u128`、`i128`、`usize` 或 `isize`。

无后缀整型字面量的类型通过类型推断确定：

- 如果整型类型可以通过程序上下文唯一确定，则无后缀整型字面量的类型即为该类型。

- 如果程序上下文对类型约束不足，则默认为 32-bit 有符号整型，即 `i32`。
- 如果程序上下文对类型约束过度，则报静态类型错误。

各种形式的整型字面量示例：

```
123;                // 类型 i32
123i32;             // 类型 i32
123u32;             // 类型 u32
123_u32;            // 类型 u32
let a: u64 = 123;    // 类型 u64

0xff;               // 类型 i32
0xff_u8;            // 类型 u8

0o70;               // 类型 i32
0o70_i16;           // 类型 i16

0b1111_1111_1001_0000; // 类型 i32
0b1111_1111_1001_0000i64; // 类型 i64
0b_____1;         // 类型 i32

0usize;             // 类型 usize
```

无效整型字面量示例：

```
// 无效后缀

0invalidSuffix;

// 数字进制错误

123AFB43;
0b0102;
0o0581;
```

```
// 类型溢出

128_i8;
256_u8;

// 二进制、十六进制、八进制的进制前缀后至少需要一个数字

0b_;
0b_____;
```

请注意，Rust 句法将 `-1i8` 视为[一元取反运算符](#)对整型字面量 `1i8` 的应用，而不是将它视为单个整型字面量。

```
词法
TUPLE_INDEX:
    INTEGER_LITERAL
```

元组索引用于引用[元组](#)、[元组结构体](#)和[元组变体](#)的字段。

元组索引直接与字面量 token 进行比较。元组索引以 `0` 开始，每个后续索引的值以十进制的 `1` 递增。因此，元组索引只能匹配十进制值，并且该值不能用 `0` 做前缀字符。

```
let example = ("dog", "cat", "horse");
let dog = example.0;
let cat = example.1;
// 下面的示例非法。
let cat = example.01; // 错误：没有 `01` 字段
let horse = example.0b10; // 错误：没有 `0b10` 字段
```

注意：元组索引可能包含一个 `INTEGER_SUFFIX`，但是这不是有效的，可能会在将来的版本中被删除。更多信息请参见

<https://github.com/rust-lang/rust/issues/60210>。

词法

FLOAT_LITERAL :

DEC_LITERAL . (紧跟着的不能是 ., _ 或者标识符)

| DEC_LITERAL FLOAT_EXPONENT

| DEC_LITERAL . DEC_LITERAL FLOAT_EXPONENT?

| DEC_LITERAL (. DEC_LITERAL)? FLOAT_EXPONENT?

FLOAT_SUFFIX

FLOAT_EXPONENT :

(e|E) (+|-)? (DEC_DIGIT|_)* DEC_DIGIT (DEC_DIGIT|_)*

FLOAT_SUFFIX :

f32 | f64

浮点型字面量有如下两种形式：

- 十进制字面量后跟句点字符 U+002E (.)。后面可选地跟着另一个十进制数字，还可以再接一个可选的指数。
- 十进制字面量后跟一个指数。

如同整型字面量，浮点型字面量也可后跟一个后缀，但在后缀之前，浮点型字面量部分不以 U+002E (.) 结尾。后缀强制设定了字面量类型。有两种有效的浮点型后缀：f32 和 f64（32-bit 和 64-bit 浮点类型），它们显式地指定了字面量的类型。

- If the program context under-constrains the type, it defaults to f64.
- If the program context over-constrains the type, it is considered a static type error. 无后缀浮点型字面量的类型通过类型推断确定：

- 如果浮点型类型可以通过程序上下文唯一确定，则无后缀浮点型字面量的类型即为该类型。
- 如果程序上下文对类型约束不足，则默认为 `f64`。
- 如果程序上下文对类型过度约束，则报静态类型错误。

各种形式的浮点型字面量示例：

```
123.0f64;      // 类型 f64
0.1f64;        // 类型 f64
0.1f32;         // 类型 f32
12E+99_f64;     // 类型 f64
5f32;           // 类型 f32
let x: f64 = 2.; // 类型 f64
```

最后一个例子稍显不同，因为不能对一个以句点结尾的浮点型字面量使用后缀句法，`2.f64` 会尝试在 `2` 上调用名为 `f64` 的方法。

浮点数的表形(representation)语义在[“和平台相关的类型”](#)中有描述。

词法

BOOLEAN_LITERAL :

 true

 | false

布尔类型有两个值，写为：`true` 和 `false`。

词法

LIFETIME_TOKEN :

 ' [IDENTIFIER_OR_KEYWORD](#)

 | '_'

LIFETIME_OR_LABEL :

' [NON_KEYWORD_IDENTIFIER](#)

生存期参数和[循环标签](#)使用 LIFETIME_OR_LABEL 类型的 token。

（尽管 LIFETIME_OR_LABEL 是 LIFETIME_TOKEN 的子集，但）任何符合 LIFETIME_TOKEN 约定的 token 也都能被上述词法分析规则所接受，比如 LIFETIME_TOKEN 类型的 token 在宏中就可以畅通无阻的使用。

为了完整起见，这里列出了（Rust 里）所有的标点符号的 symbol token。它们各自的用法和含义在链接页面中都有定义。

符号	名称	使用方法
+	Plus	算术加法 , trait约束 , 可匹配空的宏匹配器 (Macro Kleene Matcher)
-	Minus	算术减法 , 取反
*	Star	算术乘法 , 解引用 , 裸指针 , 可匹配空的宏匹配器 , use 通配符
/	Slash	算术除法
%	Percent	算术取模
^	Caret	位和逻辑异或
!	Not	位和逻辑非 , 宏调用 , 内部属性 , never型 , 否定实现
&	And	位和逻辑与 , 借用 , 引用 , 引用模式
	Or	位和逻辑或 , 闭包 , match 中的模式, if let , 和 while let
&&	AndAnd	短路与 , 借用 , 引用 , 引用模式
	OrOr	短路或 , 闭包
<<	Shl	左移位 , 嵌套泛型
>>	Shr	右移位 , 嵌套泛型
+=	PlusEq	加法及赋值
-=	MinusEq	减法及赋值

<code>*=</code>	StarEq	乘法及赋值
<code>/=</code>	SlashEq	除法及赋值
<code>%=</code>	PercentEq	取模及赋值
<code>^=</code>	CaretEq	按位异或及赋值
<code>&=</code>	AndEq	按位与及赋值
<code> =</code>	OrEq	按位或及赋值
<code><<=</code>	ShlEq	左移位及赋值
<code>>>=</code>	ShrEq	右移位及赋值 , 嵌套泛型
<code>=</code>	Eq	赋值 , 属性 , 各种类型定义
<code>==</code>	EqEq	等于
<code>!=</code>	Ne	不等于
<code>></code>	Gt	大于 , 泛型 , 路径
<code><</code>	Lt	小于 , 泛型 , 路径
<code>>=</code>	Ge	大于或等于 , 泛型
<code><=</code>	Le	小于或等于
<code>@</code>	At	子模式绑定
<code>_</code>	Underscore	通配符模式 , 自动推断型类型 , 常量项 中的非命名程序项, 外部 crate , 和 use声明
<code>.</code>	Dot	字段访问 , 元组索引
<code>..</code>	DotDot	区间 , 结构体表达式 , 模式
<code>...</code>	DotDotDot	可变参数函数 , 区间模式
<code>..=</code>	DotDotEq	闭区间 , 区间模式
<code>,</code>	Comma	各种分隔符
<code>;</code>	Semi	各种程序项和语句的结束符, 数组类型
<code>:</code>	Colon	各种分隔符
<code>::</code>	PathSep	[路径分隔符] 路径
<code>-></code>	RArrow	函数返回类型 , 闭包返回类型 , 数组指针类型
<code>=></code>	FatArrow	匹配臂 , 宏
<code>#</code>	Pound	属性

\$	Dollar	宏
?	Question	问号运算符 , 非确定性尺寸 , 可匹配空的宏匹配器

括号用于文法的各个部分，左括号必须始终与右括号配对。括号以及其内的 token 在[宏](#)中被称作“token树(token trees)”。括号有三种类型：

括号	类型
{ }	花/大括号
[]	方/中括号
()	圆/小括号